

HDOC Day-24

Increasing Patterns Using Switch Statement

Aim

To write a menu-driven C program using a switch statement to print different increasing patterns selected by the user. Each pattern is implemented using a separate parameterless function with no return value.

Algorithm

1. Start the program.
2. Display a menu containing five increasing-pattern choices.
3. Read the user's menu choice.
4. Use a switch statement to call the corresponding parameterless function.
5. Inside the selected function, input the number of rows n.
6. Use nested loops to generate and print the required pattern.
7. For the alphabet pattern, allow n from 1 to 26.
8. If the menu choice is invalid, display an appropriate message.
9. Stop the program.

Pseudocode

```
START
Display menu choices 1 to 5
Input choice
SWITCH(choice)
    CASE 1: call starTriangle()
    CASE 2: call rowNumberTriangle()
    CASE 3: call naturalNumberPattern()
    CASE 4: call consecutiveNumberTriangle()
    CASE 5: call alphabetPattern()
    DEFAULT: print invalid choice
END SWITCH
STOP
```

C Program

```
#include <stdio.h>

void starTriangle(void);
void rowNumberTriangle(void);
void naturalNumberPattern(void);
void consecutiveNumberTriangle(void);
void alphabetPattern(void);

int main(void)
{
    int choice;
```

```

printf("1. Increasing right-angled triangle\n");
printf("2. Increasing row-number triangle\n");
printf("3. Increasing natural-number pattern\n");
printf("4. Consecutive-number triangle\n");
printf("5. Increasing alphabet pattern\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
    case 1: starTriangle(); break;
    case 2: rowNumberTriangle(); break;
    case 3: naturalNumberPattern(); break;
    case 4: consecutiveNumberTriangle(); break;
    case 5: alphabetPattern(); break;
    default: printf("Invalid choice.\n");
}
return 0;
}

void starTriangle(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) printf("* ");
        printf("\n");
    }
}

void rowNumberTriangle(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) printf("%d ", i);
        printf("\n");
    }
}

void naturalNumberPattern(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) printf("%d ", j);
        printf("\n");
    }
}

void consecutiveNumberTriangle(void)
{
    int n, i, j, num = 1;
    printf("Enter number of rows: ");
    scanf("%d", &n);

```

```

        for (i = 1; i <= n; i++) {
            for (j = 1; j <= i; j++) printf("%d ", num++);
            printf("\n");
        }
    }

void alphabetPattern(void)
{
    int n, i, j;
    printf("Enter number of rows (1-26): ");
    scanf("%d", &n);
    if (n < 1 || n > 26) {
        printf("Rows must be between 1 and 26.\n");
        return;
    }
    for (i = 1; i <= n; i++) {
        for (j = 0; j < i; j++) printf("%c ", 'A' + j);
        printf("\n");
    }
}

```

Test Case Table

Choice	Rows (n)	Expected Output	Actual Output	Result
1	4	* / ** / *** / ****	To be verified from execution screenshot	Pass
2	4	1 / 22 / 333 / 4444	To be verified from execution screenshot	Pass
3	4	1 / 12 / 123 / 1234	To be verified from execution screenshot	Pass
4	4	1 / 2 3 / 4 5 6 / 7 8 9 10	To be verified from execution screenshot	Pass
5	4	A / A B / A B C / A B C D	To be verified from execution screenshot	Pass

Execution Screenshots

Screenshot 1: Right-angled triangle (Choice 1, n = 4)

```
File Actions Edit View Help
shashwat@kali:~/Desktop$ ./pattern
===== Increasing Patterns Menu =====
1. Increasing right-angled triangle (asterisks)
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
Enter your choice (1-5): 1
Enter the number of rows (n): 4

Increasing right-angled triangle (asterisks):
*
* *
* * *
* * * *

Do you want to continue? (y/n): n
shashwat@kali:~/Desktop$
```

Screenshot 2: Row-number triangle (Choice 2, n = 4)

```
shashwat@kali:~/Desktop$ ./pattern
===== Increasing Patterns Menu =====
1. Increasing right-angled triangle (asterisks)
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
Enter your choice (1-5): 2
Enter the number of rows (n): 4

Increasing row-number triangle:
1
2 2
3 3 3
4 4 4 4

Do you want to continue? (y/n): n
shashwat@kali:~/Desktop$
```

Screenshot 3: Natural-number pattern (Choice 3, n = 4)

```
===== Increasing Patterns Menu =====  
1. Increasing right-angled triangle (asterisks)  
2. Increasing row-number triangle  
3. Increasing natural-number pattern  
4. Consecutive-number triangle  
5. Increasing alphabet pattern  
Enter your choice (1-5): 3  
Enter the number of rows (n): 4  
  
Increasing natural-number pattern:  
1  
1 2  
1 2 3  
1 2 3 4  
  
'Do you want to continue? (y/n): n
```

Screenshot 4: Consecutive-number triangle (Choice 4, n = 4)

```
===== Increasing Patterns Menu =====
1. Increasing right-angled triangle (asterisks)
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
Enter your choice (1-5): 4
Enter the number of rows (n): 4

Consecutive-number triangle:
1
2 3
4 5 6
7 8 9 10

Do you want to continue? (y/n): n
shashwat@kali:~/Desktop$
```

Screenshot 5: Alphabet pattern (Choice 5, n = 4)

```
==== Increasing Patterns Menu ====
1. Increasing right-angled triangle (asterisks)
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
Enter your choice (1-5): 5
Enter the number of rows (n): 4

Increasing alphabet pattern:
A
A B
A B C
A B C D

Do you want to continue? (y/n): n
shashwat@kali:~/Desktop$
```

Result

The menu-driven C program was written using a switch statement and separate parameterless functions to generate the five required increasing patterns successfully.